Denoising diffusion probabilistic models

DDPM on the same four-component Gaussian mixture

Directed Reading Program
Florida State University

Forward noise Noise prediction Reverse sampling

The same generation problem

$$p_{\text{data}}(x) = \frac{1}{4}\sum_{j=1}^{4}\mathcal{N}(x;\mu_j,0.35^2 I_2)$$

$$\mu_1 = (-2,-2), \quad \mu_2 = (-2,2), \quad \mu_3 = (2,-2), \quad \mu_4 = (2,2)$$

- Start sampling from a two-dimensional standard Gaussian.
- Produce new points near all four centers with the right spread.
- Use fresh real samples to check the generated distribution.

What changes from the ODE lesson?

	Flow matching	DDPM here
Network predicts	Velocity	Added noise
Training states	Straight interpolations	Gaussian-corrupted data
Sampling	ODE Euler steps	Gaussian reverse transitions
Randomness	Initial point	Initial point and reverse steps
Endpoints	Noise at 0, data at 1	Data at 0, noise at T

We now implement a stochastic DDPM sampler, not an ODE flow.

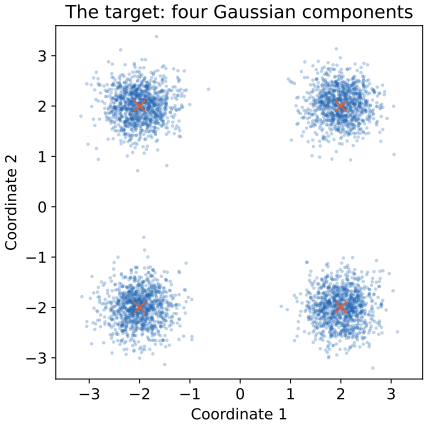
1. Create the training data

```
centers = torch.tensor([
    [-2., -2.], [-2., 2.],
    [2., -2.], [2., 2.]
])
sigma = 0.35

def sample_data(n):
    labels = torch.randint(4, (n,))
    return (centers[labels]
            + sigma * torch.randn(n, 2))

x_train = sample_data(4096)
```

Direct sampling is known here. Learning the distribution is a teaching exercise.



2. A forward step adds Gaussian noise

$$\boxed{x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \eta_t}$$

$$\eta_t \sim \mathcal{N}(0, I_2)$$

- Multiply the current point by a factor slightly below 1.
- Add fresh Gaussian noise, independently at every step.
- Repeat until the data distribution is nearly standard Gaussian.

Ho, Jain, and Abbeel (2020), Denoising Diffusion Probabilistic Models.

The noise schedule is fixed before training

```
T = 1000
beta = torch.linspace(0.0001, 0.02, T)
alpha = 1 - beta
alpha_bar = torch.cumprod(alpha, dim=0)
```

β_t is the variance of the fresh noise in one forward step.

$\alpha_t = 1 - \beta_t$ is the squared signal multiplier for that step.

Indexing: Python `k` means mathematical timestep $t = k + 1$.

`alpha_bar[0]` therefore represents $\bar{\alpha}_1$, not $\bar{\alpha}_0$.

Cumulative signal retention

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s, \qquad \bar{\alpha}_0 = 1$$

Quantity	Meaning
β_t	Variance of newly added noise
α_t	Signal retention in one step
$\bar{\alpha}_t$	Cumulative squared signal coefficient

The noise level is determined by the entire schedule up to t.

A shortcut to any forward noise level

$$\boxed{x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon}$$

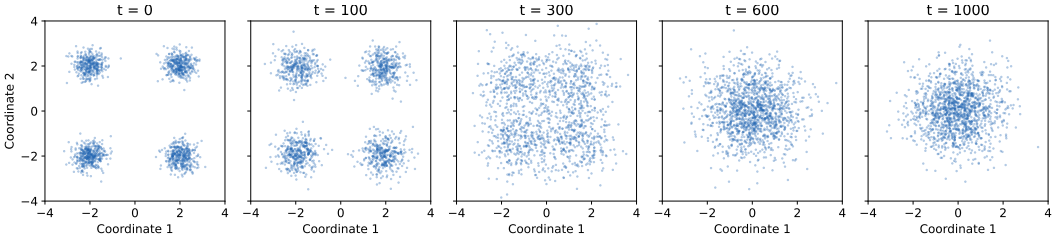
$$\varepsilon \sim \mathcal{N}(0, I_2)$$

- This samples the exact conditional marginal $q(x_t \mid x_0)$.
- ε represents aggregate noise, not only the final step's noise.
- Training can jump directly to a randomly selected timestep.

Separate shortcut draws at different times are not a Markov trajectory.

Ho, Jain, and Abbeel (2020), Denoising Diffusion Probabilistic Models.

Forward diffusion dissolves the four clusters



These panels follow one forward Markov simulation for the same 1,500 points.

Each forward step uses independent noise. The forward process has no learned parameters.

Why standard Gaussian noise is a reasonable start

$$\bar{\alpha}_T \approx 0.00004036, \qquad \sqrt{\bar{\alpha}_T} \approx 0.00635$$

The clean-data contribution is very small by $T = 1000$.

For this mixture, the four terminal component centers are near zero and their variances are near 1.

The terminal distribution is close to standard Gaussian, not exactly equal.

Shortening the schedule without checking residual signal can invalidate the initialization.

3. The network predicts noise

```
model = nn.Sequential(
    nn.Linear(3, 64),
    nn.SiLU(),
    nn.Linear(64, 64),
    nn.SiLU(),
    nn.Linear(64, 2)
)
```

Input: $(x_t^{(1)}, x_t^{(2)}, t/T)$

Output: $\varepsilon_\theta(x_t, t)$

Two noise coordinates, not a velocity.

4,546 trainable parameters.

Time in the equations is shorthand for the normalized network input.

One training example at a random timestep

```
k = torch.randint(T, (256,))
noise = torch.randn_like(x0)
a = alpha_bar[k].reshape(-1, 1)
xt = a.sqrt() * x0 + (1 - a).sqrt() * noise
t = (k + 1).reshape(-1, 1) / T
prediction = model(torch.cat([xt, t], dim=1))
```

`xt`: 256×2, `t`: 256×1, input: 256×3.

The target is `noise`, the same noise we used to construct `xt`.

4. Noise-prediction mean squared error

$$L = \frac{1}{2B} \sum_{i=1}^{B} \|\varepsilon_i - \varepsilon_\theta(x_{t_i}, t_i)\|^2$$

```
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

PyTorch averages over the batch and both coordinates.

We use uniformly sampled timesteps and the simplified DDPM objective, not the full variational likelihood bound.

Ho, Jain, and Abbeel (2020), Denoising Diffusion Probabilistic Models.

5. Training: construct noisy inputs

```
history = []
for step in range(10000):
    x0 = x_train[torch.randint(len(x_train), (256,))]
    k = torch.randint(T, (256,))
    noise = torch.randn_like(x0)
    a = alpha_bar[k].reshape(-1, 1)
    xt = a.sqrt() * x0 + (1 - a).sqrt() * noise
    t = (k + 1).reshape(-1, 1) / T
    prediction = model(torch.cat([xt, t], dim=1))
    loss = loss_fn(prediction, noise)
```

No full forward chain is needed for each training example.

Training: update the parameters

Continuation inside the same loop:

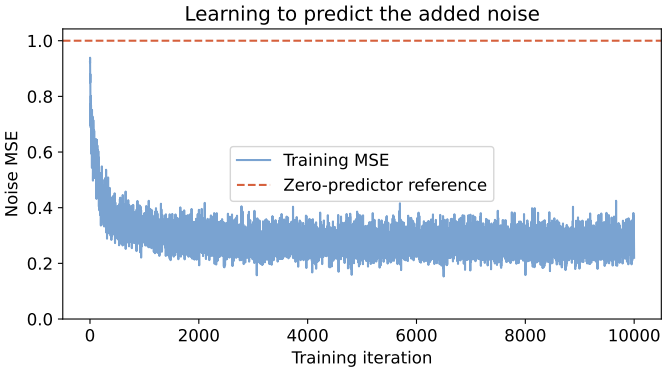
```
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    history.append(loss.item())
```

10,000 training iterations update the weights.

1,000 diffusion timesteps define the corruption and sampling chain.

Each training iteration samples examples from many diffusion timesteps.

Reading the noise-prediction loss



Always predicting zero gives expected MSE 1 per coordinate.

The best predictor averages possible noises given (x_t, t), so zero loss is not generally attainable.

Noise prediction gives a clean-data estimate

From $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$,

$$\widehat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

This explains why predicting noise helps denoising.

At high noise, the denominator is small and a one-shot estimate can be inaccurate.

DDPM instead constructs a sequence of reverse transitions.

6. The learned reverse mean

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right)$$

The network predicts noise; the schedule converts that prediction into a mean for x_{t-1}.

This is not simply $x_t - \varepsilon_\theta$. Both schedule factors matter.

The reverse transition also needs a variance.

Ho, Jain, and Abbeel (2020), Denoising Diffusion Probabilistic Models.

A fixed reverse variance

$$\tilde{\beta}_t = \beta_t \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}, \qquad \bar{\alpha}_0 = 1$$

```
alpha_bar_previous = torch.cat([torch.ones(1), alpha_bar[:-1]])
reverse_variance = beta * (1 - alpha_bar_previous) / (1 - alpha_bar)
```

This is the forward posterior variance when both x_t and clean x_0 are known.

We use it as a fixed variance for the learned Gaussian reverse transition. That transition still approximates the true reverse conditional.

$\tilde{\beta}_1 = 0$. The final sampling step adds no noise.

Ho, Jain, and Abbeel (2020), Denoising Diffusion Probabilistic Models.

Sampling: compute each reverse mean

```
def generate(z, stochastic=True):
    x = z.clone()
    path = [x.clone()]
    with torch.no_grad():
        for k in reversed(range(T)):
            t = torch.full((len(x), 1), (k + 1) / T)
            predicted_noise = model(torch.cat([x, t], dim=1))
            x = (x - beta[k] * predicted_noise
                 / (1 - alpha_bar[k]).sqrt()) / alpha[k].sqrt()
```

Python indices descend from 999 to 0. The corresponding mathematical timesteps descend from 1,000 to 1.

Sampling: add noise except at the last step

Continuation of the same function:

```
            if k > 0 and stochastic:
                x = x + reverse_variance[k].sqrt() * torch.randn_like(x)
            path.append(x.clone())
    return x, torch.stack(path)
```

$$x_{t-1} = \mu_\theta(x_t, t) + \sqrt{\tilde{\beta}_t}\, z_t \qquad (t > 1)$$

Fresh $z_t \sim \mathcal{N}(0, I_2)$ at each reverse step.

The default is stochastic DDPM. We use `False` only for an ablation.

We do not clip coordinates to image bounds such as $[-1, 1]$.

7. New Gaussian inputs generate new samples

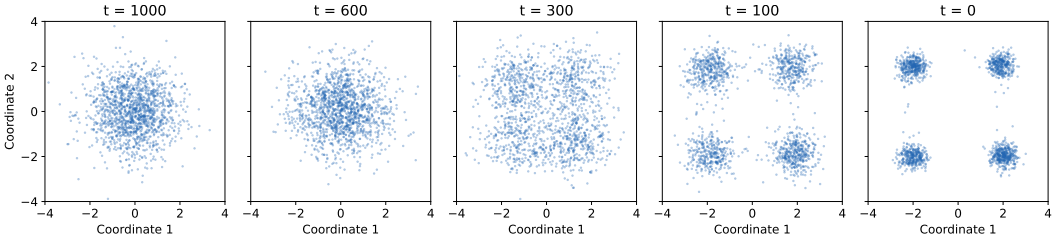
```
torch.manual_seed(2026)
model.eval()
z = torch.randn(5000, 2)
real_test = sample_data(5000)
generated, path = generate(z)
```

`real_test` is only for evaluation. It does not enter the sampler.

`path[0]` is x_T; `path[T]` is the generated x_0.

The same initial point can yield different outputs with new reverse noises.

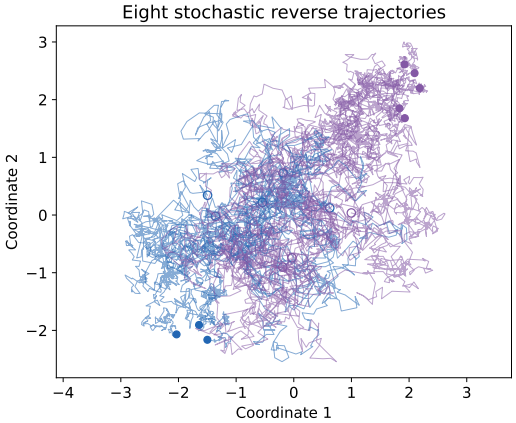
Reverse diffusion forms the four clusters



These panels follow the same 1,500 particles. Time now runs backward.

Individual steps include noise even while the whole distribution develops structure.

Stochastic reverse trajectories



Eight actual reverse paths.

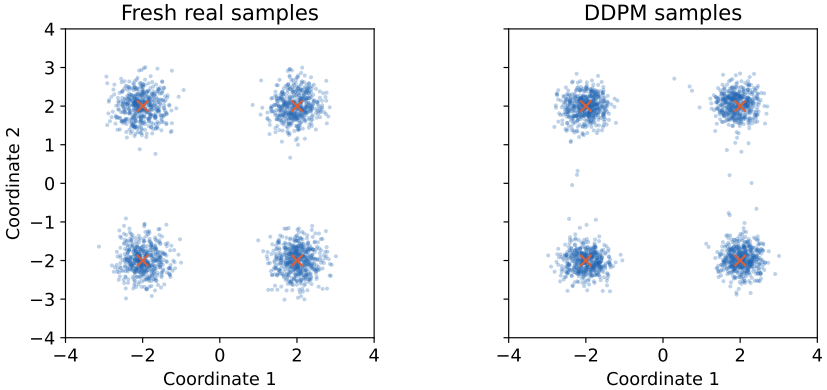
Open circles: initial noise.

Filled dots: generated endpoints.

Fresh reverse noise makes the trajectories irregular.

These are sampling paths, not time series from the target data.

Generated samples compared with fresh real data



Equal axes; 2,000 points shown per panel.

Look for all four clusters, appropriate proportions, and the correct spread.

Distribution checks on 5,000 samples

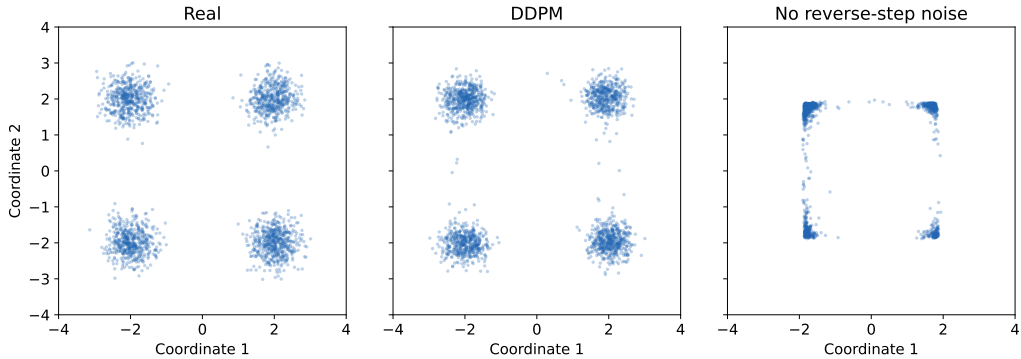
Diagnostic	Fresh real	DDPM
Nearest $(-2,-2)$ (%)	24.9000	24.7600
Nearest $(-2,2)$ (%)	24.3800	26.6800
Nearest $(2,-2)$ (%)	25.8600	24.1000
Nearest $(2,2)$ (%)	24.8600	24.4600
RMS distance to nearest center	0.4934	0.4246
Within distance 1 of a center (%)	98.4400	99.1400

Nearest-center counts check coverage; they do not identify true component labels.

Smaller radius is not automatically better. A collapsed cluster has radius zero.

These diagnostics do not certify a full distribution match.

Experiment: removing reverse-step noise



RMS radius: real 0.4934, DDPM 0.4246, mean-only 0.4292.

Same initial noise; the mean-only clusters shift inward and change shape.

Similar RMS radii do not imply similar distributions.

This mean-only chain is not DDIM or a probability-flow ODE.

Experiments and further reading

1. Change the seed or reduce training to 2,000 iterations.
2. Make the target clusters narrower and examine their learned widths.
3. Shorten T and check $\bar{\alpha}_T$ before retraining.
4. Keep the same starting point and change the reverse noise draws.

Do not simply skip timesteps with the original one-step coefficients.

Use fresh test data after tuning. These examples are not a controlled ranking of DDPM versus ODE generation.

References: Ho et al. (2020), DDPM; authors' implementation; Song et al., DDIM.

Companion notebook: `learning_ddpm.ipynb`